1 Blackbox Description

I treat ssdeep as a command-line file similarity tool. A user supplies one or more files, directories, or previously generated signature files. ssdeep then generates fuzzy signatures, compares signatures, or groups similar files, depending on the selected command-line options.

A conventional cryptographic hash answers a binary question: either two files contain exactly the same data or they do not. Even a one-byte modification normally produces a completely different cryptographic hash. ssdeep instead uses context-triggered piecewise hashing, which developers commonly call fuzzy hashing. This technique allows the program to estimate how much content two files share.

A user gives ssdeep one or more files or directories. The program reads each file and creates a fuzzy hash that represents its contents. It can print these hashes for later use or compare them to find similar files. When ssdeep compares two fuzzy hashes, it produces a similarity score between 0 and 100. A higher score indicates a stronger similarity.

For example, a user could give ssdeep a directory containing several versions of the same document. Although the files may contain additions or edits, ssdeep may still identify them as related. This differs from a normal cryptographic hash, which only supports exact matching.

I used 4 main parts of the original program. `main.cpp` reads the command-line options and selects the requested operation. `dig.cpp` finds the files to process, including files inside directories. `engine.cpp` opens each file and calls the hashing code in `fuzzy.c` to produce its signature. `match.cpp` manages signature comparisons and handles the resulting matches.

For file hashing, the original program processes files sequentially. It finds a file, calculates its signature, and handles the result before moving to the next file. For comparison, ssdeep can use signatures generated from the input files or read previously saved signatures from a text file. The comparison code passes pairs of signatures to `fuzzy_compare`, then prints or groups matches according to the selected options. It also performs these comparisons sequentially.

2 Potential Parallelism

I found two main opportunities for parallelism in ssdeep: hashing different files at the same time and comparing different pairs of file signatures at the same time.

Figure 1 shows how ssdeep processes files from a directory. The loop reads one directory entry, creates its full path, and calls `process_normal`. This function checks the path and calls `hash_file` when the path represents an eligible file. The call finishes before the loop moves to the next directory entry, so the original program hashes files sequentially.

Currently, `new_file` stores the full path of the current directory entry. The loop reuses this variable for every entry. If I ran the loop iterations concurrently without changing the code, one thread could overwrite the path while another thread still uses it. This race condition could cause a thread to process the wrong file.

I first collect the paths into a list so that each OpenMP thread receives a different stored path. Each thread will calculate its file's hash and save the result separately. Since the current code prints each result immediately, the output gets delayed until the parallel loop finishes. The main thread will then print the results in their original order.

```
205   while ((entry = _treaddir(current_dir)) != NULL)
206   {
207     if (is_special_dir(entry->d_name))
208       continue;
209
210     _sntprintf(new_file,SSDEEP_PATH_MAX,_TEXT("%s%c%s"),
211           fn,DIR_SEPARATOR,entry->d_name);
212
213     return_value = process_normal(s,new_file);
214   }
```

```
348   bool process_normal(state *s, TCHAR *fn)
349   {
350     clean_name(s,fn);
351
352     if (should_hash(s,fn))
353       return (hash_file(s,fn));
354
355     return false;
356   }
```

Figure 1: The sequential file-processing path.

The second opportunity appears in the signature comparison loop shown in Figure 2. During each iteration, `fuzzy_compare` compares the current signature with one signature from `all_files` and returns a similarity score. Each comparison only needs the two signatures involved, so calculating one score does not depend on any other comparison.

When `fuzzy_compare` finds a match, the loop calls `handle_match`. This function prints the match or places the files into a group. If several threads call it at the same time, they could mix their output or change the same group. Therefore, it's better to let each thread calculate and save its scores. After all comparisons finish, the main thread will process the saved results. Doing it this way gives more parallel work as the number of files increases.

I decided not to divide a single file between threads. As ssdeep reads a file, it keeps a running hashing state based on the bytes it has already processed. It uses this state to process the next byte and decide where to divide the file into pieces. A thread starting in the middle of the file would not have the state produced from the earlier bytes, so it could not calculate that section correctly on its own. Combining separately calculated sections could therefore produce a different fuzzy hash from the original sequential program.

File hashing can use more threads when the dataset contains more files, although storage speed

may eventually become a bottleneck. Signature comparison can give more scalable parallelism because the number of file pairs increases quickly as the dataset grows. I don't actually need to replace the existing hashing or comparison algorithms. Instead, I will reorganize the program so that it collects the input, performs the independent work in parallel, and then handles the results.

```
std::vector<Filedata* >::const_iterator it;
for (it = s->all_files.begin() ; it != s->all_files.end() ; ++it)
{
  // When in pretty mode, we still want to avoid printing
  // A matches A (100).
  if (s->mode & mode_match_pretty)
  {
    if (!(_tcsncmp(f->get_filename(),
          (*it)->get_filename(),
          std::max(fn_len,_tcslen((*it)->get_filename())))) &&
  (f->get_signature() == (*it)->get_signature()))
    {
// Unless these results from different matching files (such as
// what happens in sigcompare mode). That being said, we have to
// be careful to avoid NULL values such as when working in
// normal pretty print mode.
if (!f->has_match_file() ||
    f->get_match_file() == (*it)->get_match_file())
  continue;
    }
  }

  int score =  fuzzy_compare(f->get_signature().c_str(),
               (*it)->get_signature().c_str());
  if (-1 == score)
    print_error(s, "%s: Bad hashes in comparison", __progname);
  else
  {
    if (score > s->threshold || MODE(mode_display_all))
    {
handle_match(s,f,(*it),score);
status = true;
    }
  }
}
```

Figure 2: The sequential signature-comparison loop.

3 Mapping and Synchronisation

I used OpenMP threads to distribute work between the CPU cores. For file hashing, I first collect the file paths into a list. I then use an OpenMP `parallel for` loop with dynamic scheduling to distribute the files between the available threads. Each loop iteration hashes one file and stores the result in a separate position in a results list.

For signature comparison, I process the stored signatures in batches of up to 1,024. I use another OpenMP `parallel for` loop with dynamic scheduling to distribute the signatures in each batch between the available threads. When a thread finishes processing one signature, it can take the next unprocessed signature from the batch. Each thread compares its assigned signatures against all the stored signatures and saves the resulting scores.

I tested the batch size on the 24-core Oracle VM using 65,536 signatures. I tested batch sizes of 256, 512, 1,024, 8,192, and 65,536. I performed one warm-up run for each batch size, followed by three measured runs in a randomized order.

Figure 3 shows that a batch size of 8,192 was the fastest, completing the test in 104.88 seconds while using 2,071 MiB of memory. A batch size of 1,024 completed the test in 106.81 seconds while using only 279 MiB.

Using a batch size of 65,536 placed every signature in one batch. This was equivalent to storing the complete set of comparison scores in memory. It finished the test in 105.19 seconds but needed 16,409 MiB. Therefore, the batch size of 1,024 was only 1.5% slower while using 98% less peak memory. Smaller batch sizes reduced memory further but increased the execution time slightly. I chose 1,024 because it kept performance pretty close to the fastest results without storing an unnecessarily large number of scores in memory.

I prevent multiple threads from printing results or updating clustering information at the same time. These actions could otherwise produce incorrect or disordered output. Each thread only writes to its own position in the results list while the parallel loop runs. OpenMP provides an automatic barrier at the end of each `parallel for` loop, which means the program waits until every thread finishes its assigned work. The main thread then processes the saved results and prints them in their original order. Therefore, I did not need to add explicit locks or critical sections.

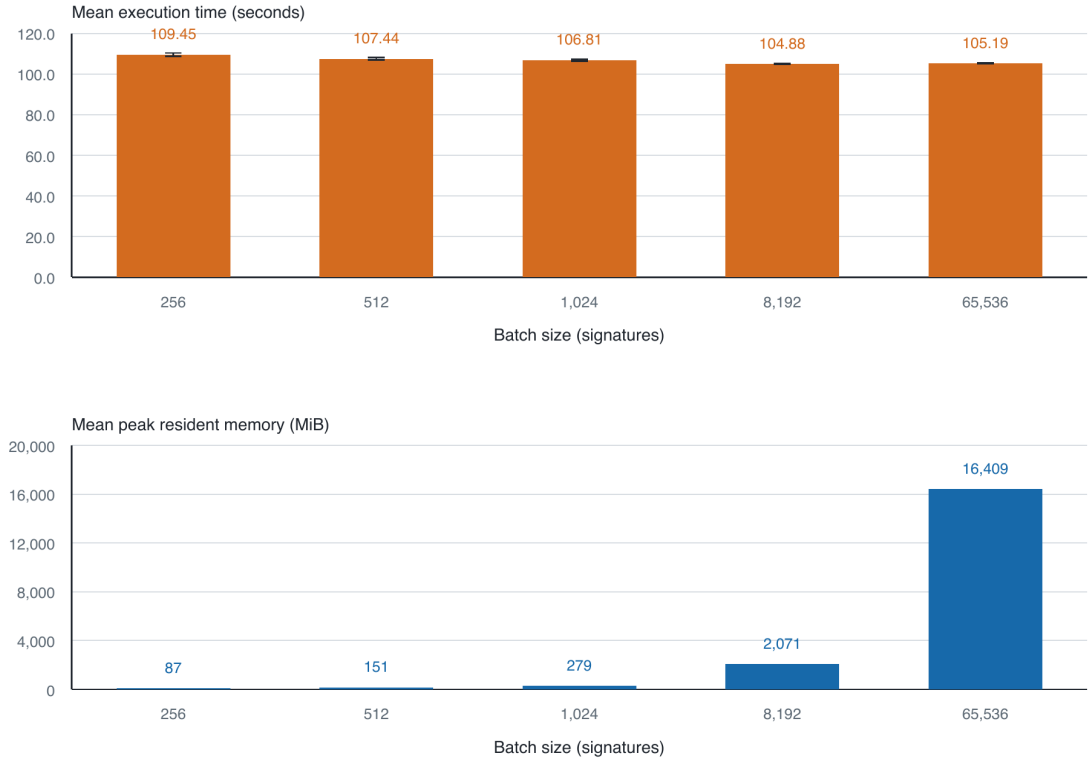


Figure 3: Mean execution time and mean peak memory usage for different batch sizes using 65,536 signatures and 24 threads. The execution-time error bars show the standard deviation across three measured runs.

4 Results

I performed the final thread-count tests on an Oracle Cloud VM with 24 Arm cores and 32 GB of memory. Each core supported one hardware thread. I used Ubuntu 24.04 and compiled both versions in Release mode using GCC 13.3 and GNU OpenMP.

For file hashing, I used 240 files of 4 MiB each, giving a total size of 960 MiB. For signature comparison, I used 4,096 signatures. I tested the modified program with 1, 2, 4, 8, 12, 16, 20, and 24 threads. I did two warm up runs and then recorded ten measured runs for each configuration.

Each execution time shows the mean of the measured runs. The value after the ± symbol is the standard deviation, which shows how much the execution times varied between runs.

Table 1 shows the timing results, while Figure 4 shows the resulting speedup.

Version	Threads	Hashing time (s)	Hashing speedup	Comparison time (s)	Comparison speedup
Original	1	9.055 ± 0.073	1.000	10.482 ± 0.677	1.000
Modified	1	9.085 ± 0.006	0.997	9.298 ± 0.552	1.127
Modified	2	4.551 ± 0.004	1.990	4.604 ± 0.099	2.277
Modified	4	2.287 ± 0.007	3.960	2.316 ± 0.057	4.526
Modified	8	1.144 ± 0.005	7.913	1.176 ± 0.012	8.915
Modified	12	0.769 ± 0.001	11.783	0.801 ± 0.007	13.088
Modified	16	0.577 ± 0.001	15.694	0.618 ± 0.008	16.960
Modified	20	0.466 ± 0.0003	19.421	0.503 ± 0.005	20.820
Modified	24	0.390 ± 0.001	23.210	0.428 ± 0.002	24.487

Table 1: Oracle VM execution times and speedup from ten measured runs. The values after ± show the standard deviation.

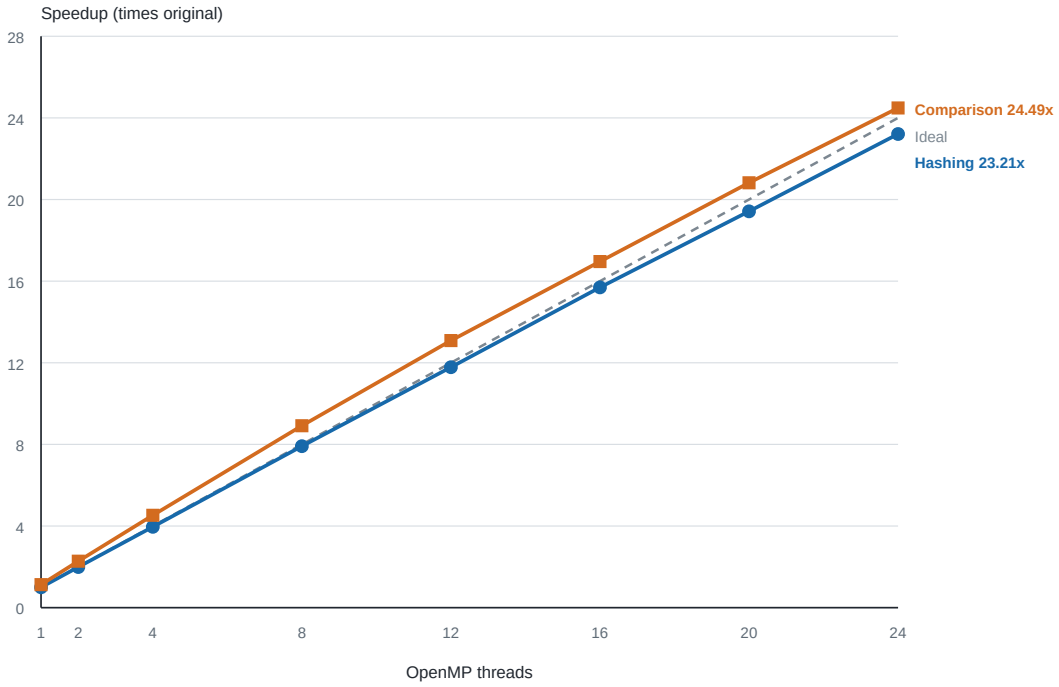


Figure 4: Speedup on the 24-core Oracle Cloud VM relative to the original sequential version.

The Oracle VM produced close to linear scaling across its 24 cores. With 24 threads, file hashing achieved a speedup of 23.21 and signature comparison achieved an overall speedup of 24.49.

The comparison speedup was slightly higher than the number of cores because I calculated it relative to the original program. The modified comparison code was already 1.127 times faster with one thread because it avoided repeated signature copies. When I compared the modified version using one and 24 threads, the speedup was 21.72.

Linux perf showed that the original program used approximately one CPU core. During the 24-thread tests, the modified file-hashing code used an average of 23.4 cores, while signature comparison used an average of 22.1 cores. This confirmed that the OpenMP loops distributed work across the available cores.

I used Linux `perf` to find where each test spent its CPU time. I profiled the original version with one thread and the modified version with 24 threads, running each test three times. The percentages show each function's average share of CPU-cycle samples within its test.

In the original hashing test, `fuzzy_update`, which processes file bytes, accounted for 99.79% of samples. It accounted for 99.63% in the modified test. File byte processing was the main CPU work in both versions.

When ssdeep compares two signatures, it uses `copy_eliminate_sequences` to remove long runs of repeated characters. It then uses `score_strings` to calculate a similarity score from 0 to 100. In the original comparison test, these functions accounted for 53.78% and 18.60% of CPU-cycle samples, respectively. In the modified test, they accounted for 58.56% and 20.21%. This shows where much of the comparison test's CPU work happened.

5 Correctness

I wrote a Python test that runs the original and modified programs on the same files. The test creates 160 files by copying three different files from the original ssdeep source code. This gives it both identical copies and files with different contents.

I tested recursive file hashing, directory comparison, pretty comparison, and comparison using a saved signature file. For each operation, the script saves the original program's standard output. It then runs the modified program with 1, 2, 4, 8, 10, 12, 16, 20, and 24 threads and compares its standard output with the original byte-for-byte. The script also fails if either program exits with an error.

This check includes the hashes, filenames, similarity scores, formatting, and output order. Every run on this dataset matched the original program exactly, even though threads could finish their calculations in a different order.

6 Compilers, Software, Tools and Techniques

II wrote the parallel changes in C++11 and used OpenMP to run work on multiple CPU cores. On the Oracle VM, I compiled the program using GCC 13.3 and GNU OpenMP. I used CMake to build the original and modified versions with the same Release settings so that I could compare them fairly.

I used OpenMP parallel for loops for both file hashing and signature comparison. I used dynamic scheduling so that a thread could take another file or signature after finishing its current work. This helped when individual tasks took different amounts of time. For signature comparison, I processed the signatures in batches of 1,024 to avoid storing every comparison result in memory at once.

Each thread saved its result in a separate position in a results list. OpenMP waited for every thread to finish before the program continued. The main thread then processed and printed the saved results in their original order. This approach avoided the need for locks around the program's output and clustering code.

I wrote some Python scripts to make repeatable test data, run the timing tests, calculate the mean, standard deviation and speedup, and compare the output of the original and modified programs. I used Linux `perf` to look at where the program spent its processing time and verify that the modified version used multiple cores.

7 Barriers

The first barrier was that the original program calculated a result and acted on it right away. After comparing two signatures, it called `handle_match` if their similarity score was high enough. That function prints which files matched or adds them to a group of similar files. If several threads called it at once, their printed lines could get mixed up, or they could try to update the same group.

I changed the threads to calculate and save scores only. After the threads finish each batch, the main thread reads the saved scores in their original order and prints matches or updates the groups. This let me run the comparisons concurrently without changing the order of the output.

Figure 5 shows how I separated score calculation from match handling. In the modified code, `calculate_comparison_scores` saves the scores, while `handle_comparison_scores` later calls `handle_match` from the main thread.

Before: calculate and handle each match immediately

```
317       int score =  fuzzy_compare(f->get_signature().c_str(),
318                                  (*it)->get_signature().c_str());
319       if (-1 == score)
320         print_error(s, "%s: Bad hashes in comparison", __progname);
321       else
322       {
323         if (score > s->threshold || MODE(mode_display_all))
324         {
325     handle_match(s,f,(*it),score);
326     status = true;
327         }
328       }
```

After: calculate scores in parallel, then handle them in order

```
406       std::vector<std::vector<int> > batch_scores(batch_count);
407
408   #pragma omp parallel for schedule(dynamic)
409       for (long long offset = 0; offset < (long long)batch_count; ++offset)
410       {
411         Filedata *file = s->all_files[batch_start + (size_t)offset];
412         batch_scores[(size_t)offset] =
413     calculate_comparison_scores(s, file, false);
414       }
415
416       for (size_t offset = 0; offset < batch_count; ++offset)
417       {
418         Filedata *file = s->all_files[batch_start + offset];
419         const bool status =
420     handle_comparison_scores(s, file, batch_scores[offset]);
```

Figure 5: The original and modified signature-comparison code.

File hashing had a similar problem. The original directory loop reused `new_file` to hold each file's path, and the hashing code printed each result as soon as it finished. I kept the directory scan sequential and copied each path into a list. Each thread then hashed a different file and saved its result separately. Once all the threads finished, the main thread printed the hashes in their original order.

I also had to choose how much work to give each thread. I chose a whole file as one hashing task because ssdeep processes the bytes within a file in order, using the hash state produced by earlier bytes. For comparison, I made one task compare a stored signature with the other stored signatures. I used OpenMP's dynamic scheduling so a thread could take another task when it finished its current one.

The final barrier was memory use during comparison. I tested a batch containing all 65,536 signatures, which kept all its calculated scores in memory until the parallel work finished. It used 16,409 MiB of peak memory. I changed the program to process at most 1,024 signatures per batch, so it only needed to keep one batch's scores at a time. Peak memory fell to 279 MiB. Execution time increased from 105.19 to 106.81 seconds, a 1.5% cost for a 98% reduction in peak memory.

8 Code Explanation

I changed 6 source files: `dig.cpp`, `engine.cpp`, `filedata.h`, `main.cpp`, `match.cpp`, and `ssdeep.h`. The modified files contain 1,968 lines, compared with 1,814 lines in the original version. The changes added 229 lines and removed 75, leaving a net increase of 154 lines.

For file hashing, I changed the program so that it first collects the file paths into a list. I added an OpenMP loop in `engine.cpp` that distributes these files between the threads. Each thread hashes a different file and saves its result. The main thread then prints the saved results in their original order.

For signature comparison, I separated score calculation from output. The OpenMP threads calculate and save the scores, and the main thread prints matches and updates clustering information after the parallel work finishes. I also added batching so that the program processes up to 1,024 signatures at a time instead of keeping every comparison score in memory.

The original code copied a signature every time it used it in a comparison. I changed `filedata.h` so that the comparison code reads the existing signature instead, avoiding repeated copies.

9 Reflection

The parallelization was successful. The new program had the same output as the original in every correctness test. On the 24-core Oracle VM, file hashing achieved a speedup of 23.21 relative to the original program, while signature comparison achieved a speedup of 24.49. The comparison result exceeded 24 because the modified code also avoided repeated signature copies. When I compared the modified version using one and 24 threads, OpenMP produced a parallel speedup of 21.72.

I learned that parallelizing a program involves more than adding an OpenMP instruction to a loop. I first had to separate calculations from printing, give each thread its own input and result position, and then also maintain the original output order. I also learned that reducing execution time can create another problem. Saving more comparison results at once slightly improved performance but greatly increased memory usage. Batching allowed me to control this memory usage without losing much performance.

I don't think that this is the best possible version for every computer or dataset. In my comparison between my Mac's performance and the Oracle VM, it was clear that the speedup gained also depends on the hardware of the machine.

If I repeated the project, I would use larger and more varied datasets from the beginning. I initially tested only on my Mac, where the speedup curve began to level off as I increased the thread count. This led me to believe that the program had reached diminishing returns. However, the Oracle VM continued to achieve close to linear speedup across 24 identical cores.

Given more time and resources, I would test Oracle VMs with more CPU cores, more memory, and larger datasets to find where this improvement eventually begins to level off. I would also test on both systems earlier and close other applications before every benchmark. I had to repeat some of my initial Mac tests because I used my browser while the tests ran, which could have affected the results. Finally, I would try choosing the batch size automatically based on the number of signatures and the memory available on the machine.